

実験ログ: 地形 (HeightField)

2026-09-24 | 収録 8 件

Houdini 研究部で、hython で実際に測った実験の記録。数字はどれも測った値。

収録している実験

- ・ 平面で地形を作り、指標を作り直す
- ・ scatter と copy to points — 地面に物を生やす
- ・ HeightField で地形を作る — 高さを面ではなく数の並びで持つ
- ・ 地形の上に生やし分ける — 浸食が残した情報で木の場所を決める
- ・ 流れの向きに草を寝かせる — flowdir は x と y が入れ替わっている
- ・ heightfield_noise の Amplitude は高さの幅ではない — 幅は Amplitude の 23~29%、Center Noise を切ると Amplitude の半分だけ上がる
- ・ heightfield_erode は土の量を保たない — 40 フレームで height の平均が 5 下がり、削れた分の大半は地形から消える
- ・ heightfield_noise の高さの幅は、地面に入る模様の数で決まる — Element Size を地面の 1/20 にすると Amplitude の 6 割

平面で地形を作り、指標を作り直す

2026-09-24 | Houdini 21.0.700 | /obj/grid_terrain | ノード 12 / 接続 8

何を作ったか

これまでの指標は「中心からの距離の差」で、球体でしか使えず、点数をまたいだ比較もできなかった（006でその限界にぶつかった）。平面なら高さの差をそのまま測れる。さらに点の間隔で割れば勾配（傾き）になるので、これなら解像度によらない量になるはずだと考えた。

結果として、その期待は外れた。勾配は解像度を上げるほど増え続ける。ただしこれは失敗ではなく、狙っていたものより役に立つ道具になった。勾配が増えなくなる点が「その形を表現するのに十分な解像度」を教えてくれる。

あわせて、004で「平面ならもっと差が出るかもしれない」と書いた fractal の種類も確かめた。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
grid_20	grid	cols = 20 rows = 20
mt_20	mountain::2.0	elementsize = 2.0
grid_40	grid	cols = 40 rows = 40
mt_40	mountain::2.0	elementsize = 2.0
grid_80	grid	cols = 80 rows = 80
mt_80	mountain::2.0	elementsize = 2.0
grid_160	grid	cols = 160 rows = 160
mt_160	mountain::2.0	elementsize = 2.0
frac_none	mountain::2.0	elementsize = 2.0 fractal = none
frac_fBm	mountain::2.0	elementsize = 2.0 fractal = fBm
frac_mfT	mountain::2.0	elementsize = 2.0 fractal = mfT
frac_hmfT	mountain::2.0	elementsize = 2.0

解像度を変える（elementsize 2.0、height 1.0 固定）

10×10 の grid の分割数を 20 から 160 まで上げた。点の間隔はそれに応じて 0.53 から 0.06 まで細くなる。

分割数	点数	点の間隔	平均の高さ	最大の高さ	平均勾配
20	400	0.5263	0.07919	0.29407	0.13595

分割数	点数	点の間隔	平均の高さ	最大の高さ	平均勾配
40	1600	0.2564	0.08187	0.32680	0.16634
80	6400	0.1266	0.08183	0.32995	0.18745
160	25600	0.0629	0.08196	0.33513	0.20430

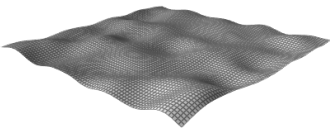
オクターブ数ごとに、勾配がどこで止まるか

各行が1つの解像度。オクターブ数を増やすほど細かい成分が入るので、それを表現しきるのに必要な解像度も上がる。数字が横に並んで止まっていれば、その解像度で足りているということ。

分割数	oct = 1	oct = 2	oct = 4	oct = 8	oct=8 の 80→160 増加率
20	0.12897	0.13673	0.13593	0.13595	—
40	0.14205	0.15845	0.16659	0.16634	—
80	0.14492	0.16561	0.18604	0.18745	—
160	0.14570	0.16752	0.19605	0.20430	+9.0%
収束したか	した (+0.5%)	ほぼした (+1.2%)	まだ (+5.4%)	まだ (+9.0%)	

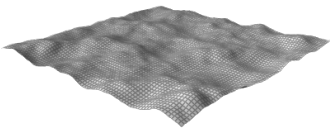
fractal の4種類（分割数80）

004では球体で比べて差が小さかった（細かさ 0.01350~0.01786）。平面では勾配 0.12216 から 0.19064 まで1.6倍の開きが出た。004で立てた「平面ならもっと差が出るかもしれない」という見込みは当たっていた。



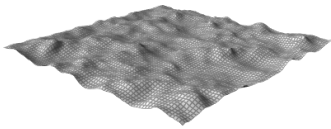
Houdini

None 勾配 0.12216 — 最も滑らか



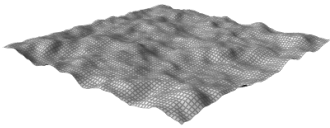
Houdini

Terrain 勾配 0.14766



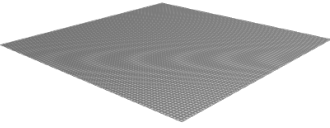
Houdini

Hybrid Terrain（既定） 勾配 0.18745



Houdini

Standard (fBm) 勾配 0.19064 — 最も粗い



Houdini

基準: ノイズなしの grid 6400点 / 6400面

fractal	平均の高さ	最大の高さ	平均勾配	上向きの点の割合
None	0.08386	0.39362	0.12216	44.4%
Terrain	0.08257	0.38162	0.14766	43.6%

fractal	平均の高さ	最大の高さ	平均勾配	上向きの点の割合
Hybrid Terrain (既定)	0.08183	0.32995	0.18745	42.3%
Standard (fBm)	0.08269	0.37818	0.19064	45.4%

わかったこと

- ・ grid SOP の既定は 10×10 のサイズ、分割は 10×10、向きは ZX 平面（水平）、種類はポリゴン。水平に置かれるので、mountain の押し出しは真上（Y方向）になる。だから高さの差をそのまま測れる。
- ・ 狙いは外れた。勾配は解像度によらない量にはならず、分割数を上げるほど増え続けた（0.13595 → 0.16634 → 0.18745 → 0.20430）。fBm 系のノイズは細かい成分を何段も持っているので、細かく刻むほど急な斜面が新たに見えてくるため。
- ・ ただしこの性質のおかげで、別のことが分かるようになった。オクターブ数ごとに勾配の伸びを見ると、oct = 1 は 80分割で収束し（80→160 で +0.5%）、oct = 2 はほぼ収束（+1.2%）、oct = 4 はまだ伸び（+5.4%）、oct = 8 もまだ伸びている（+9.0%）。つまり勾配が増えなくなったら、その解像度でその形を表現しきれているという判定に使える。
- ・ 低い解像度では余分なオクターブがまったく見えない。20分割では oct = 2 / 4 / 8 がほぼ同じ値（0.13673 / 0.13593 / 0.13595）になり、oct = 2 がわずかに最大という順序の逆転まで起きている。004で球体を使って得た結論が平面でも再現し、しかも解像度との関係まで数字で見えた。
- ・ 高さの平均は40分割以降ほぼ一定（0.0819前後）。振幅は低い解像度でもすぐ正しくなるが、細かさはずっと遅れて収束する。形が正しく見えるのに必要な解像度は、高さが合う解像度よりはるかに高いということ。点数を決めるときは高さではなく細かさを基準にする必要がある。
- ・ 最大の高さも解像度とともに増える（0.29407 → 0.33513）。これも同じ理由で、細かく刻むほどノイズの尖った部分を拾えるようになるため。005・006で使った「最大のずれ」も同じ性質を持っていたことになる。
- ・ fractal の種類は平面で明確に差が出た。滑らかな順に None（0.12216）、Terrain（0.14766）、Hybrid Terrain（0.18745）、Standard fBm（0.19064）。一方で平均の高さはどれも 0.0818~0.0839 でほぼ同じ。fractal は振幅ではなく細かさを調えるパラメータだと確定した。
- ・ None が最も滑らかなのは筋が通っている。層を重ねない単一のノイズなので、細かい成分が入らない。004で「rough = 0 は fractal = None と完全に一致する」と分かったことと整合する。
- ・ カメラのフレーミングを直した。以前はバウンディングボックスを立方体とみなして距離を決めていたため、平たい地形が画面の4分の1ほどにしか写らなかった。8つの隅をカメラ空間に投影して必要な距離を解く方式に変更し、対象が画面いっぱい収まるようになった。地形を扱う以上、これは必須の修正だった。

次にやること

- ・ elementsize・オクターブ数・必要な分割数の関係を式にまとめる。「この設定ならこれだけの点数が必要」と事前に見積もれるようにする
- ・ 地形専用の heightfield ノード群と、この grid + mountain のやり方を比べる
- ・ パラメータの調査はここで区切り、scatter と copy to points に進む

scatter と copy to points — 地面に物を生やす

2026-09-24 | Houdini 21.0.700 | /obj/scatter_copy | ノード 12 / 接続 13

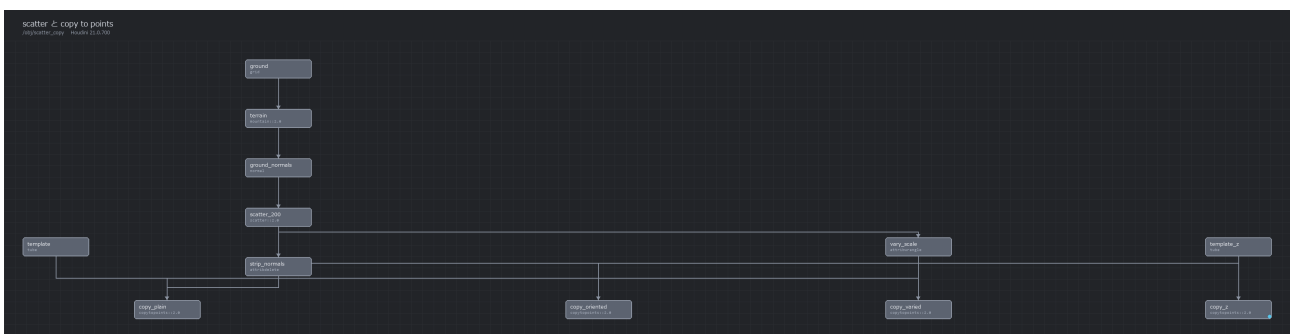
何を作ったか

面の上に点をばらまき、その点ごとにジオメトリを複製する。プロシージャルモデリングの中核パターンで、草や岩や建物を並べるのは全部これで行う。

測ったのは3つ。散布点の数は指定どおりになるか、複製後の点数はどう増えるか、そして複製が地面の傾きに沿って回るかどうか。

3つめで重要な落とし穴が見つかった。複製は確かに回るのだが、回り方が想像と違う。数字で確定させた。

ノードの接続



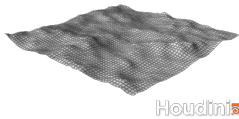
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
ground	grid	cols = 60 rows = 60
terrain	mountain::2.0	elementsiz = 3.0 height = 1.2
ground_normals	normal	(変更なし)
scatter_200	scatter::2.0	npts = 200
template	tube	cap = 1 cols = 10 height = 0.9 rad1 = 0.0 rad2 = 0.18 type = 1
strip_normals	attribdelete	ptdel = N
copy_plain	copytopoints::2.0	(変更なし)
copy_oriented	copytopoints::2.0	(変更なし)
vary_scale	attribwrangle	snippet = @pscale = fit01(rand(@ptnum * 7.3), 0.35, 1.4);
copy_varied	copytopoints::2.0	(変更なし)

ノード	タイプ	デフォルトから変更したパラメータ
template_z	tube	cap = 1 cols = 10 height = 0.9 orient = 2 rad1 = 0.0 rad2 = 0.18 type = 1
copy_z	copytopoints::2.0	(変更なし)

複製の向きと大きさ

地面は 10×10 の grid（60分割）に mountain をかけたもの。散布点は200個、テンプレートは20点の円錐。3つの条件で複製した。



地面 grid 60分割 + mountain



法線なしの点に複製
向きのばらつき 0.00度



法線ありの点に複製 + pscale
向きのばらつき 10.84度



テンプレートをZ軸向きにした場合 N と完全に一致する

条件	複製後の点数	複製数	予想点数	向きのばらつき（平均 / 最大）	大きさの幅
法線なし	4000	200	4000	0.00度 / 0.00度	0.45 固定
法線あり	4000	200	4000	10.84度 / 31.69度	0.45 固定
法線あり + pscale	4000	200	4000	10.84度 / 31.69度	0.16 ~ 0.63

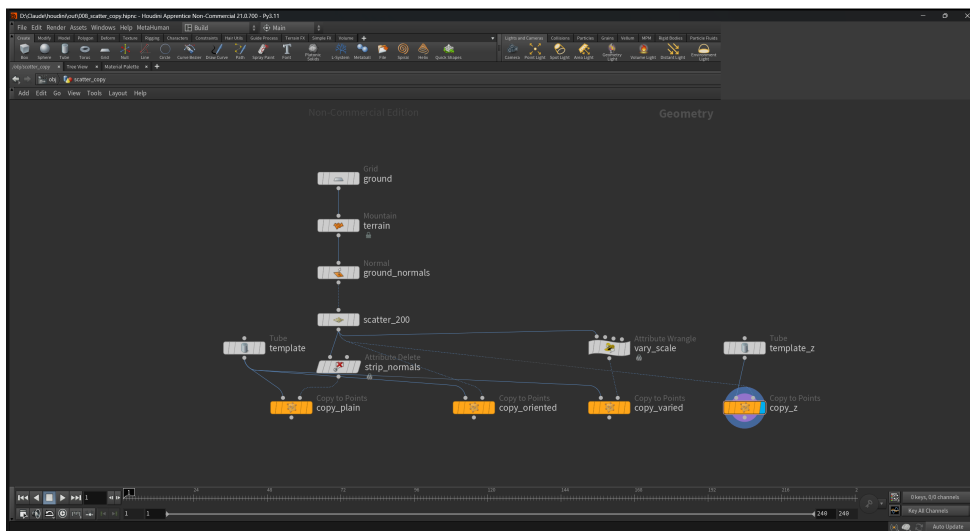
N はテンプレートのどの軸に対応するのか

複製が地面に沿って回ってはいるのに、円錐が横倒しに見えた。そこで、各複製の軸と、その複製が乗っている点の N との角度を全200個について測った。結果はばらつきゼロで、完全に一方の答えを示した。

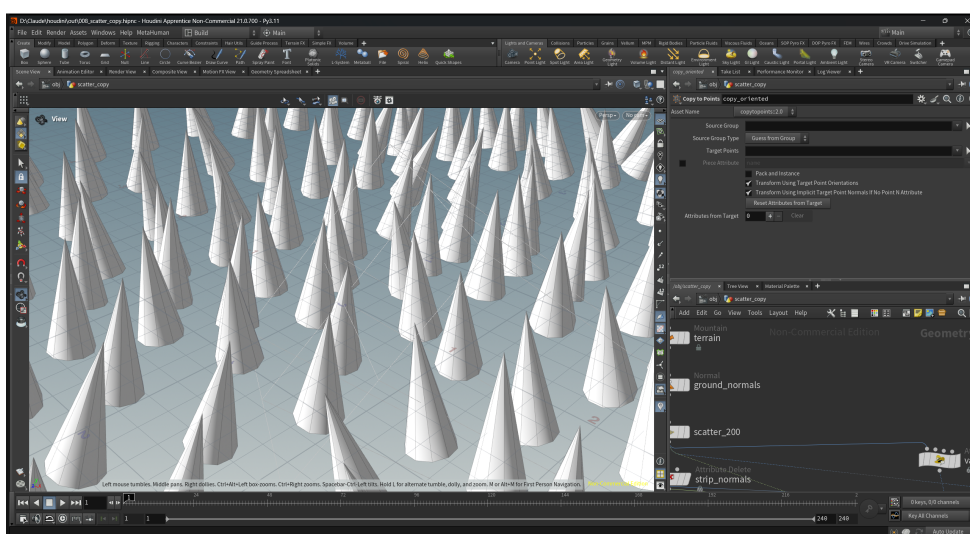
テンプレートの向き	N との角度（平均）	最小	最大
Y軸（tube の既定）	90.00度	90.00度	90.00度
Z軸	0.00度	0.00度	0.00度

実際の Houdini の画面

自動生成の接続図と違い、これは起動中の Houdini を外から操作して撮った本物の画面。自分の Houdini で同じものを開いたときに何が見えるかがそのまま分かる。パラメータ側に写っている「Transform Using Target Point Orientations」が、この実験で測った「N で回転するかどうか」の正体。



ネットワークエディタ。scatter_200 から4つの Copy to Points に分岐している



copy_oriented のパラメータと、その結果のビューポート

scatter の点数は指定どおりか

指定した数	実際の点数	一致
50	50	はい
200	200	はい
800	800	はい

わかったこと

- ・ scatter の npts は正確に守られる。50 / 200 / 800 を指定して、実際の点数も 50 / 200 / 800 になった。おおよその数ではなく、指定した数ちょうどが出る。
- ・ scatter の出力は P だけでなく N も持っている。元の面の法線が引き継がれるため。この N があるかないかで、次の copy to points の挙動が変わる。
- ・ copy to points の点数は掛け算そのまま。散布点200個 × テンプレート20点 = 4000点。予想と実測が完全に一致した。テンプレートが重いと一気に破綻するので、散布数とテンプレートの点数は掛け算で効くと覚えておく。

- ・ N を持たない点に複製すると、複製は一切回転しない。向きのばらつきは平均も最大も 0.00度。200個すべてが同じ向きに立つ。

- ・ N を持つ点に複製すると、点ごとに回転する。向きのばらつきは平均 10.84度、最大 31.69度で、これは地面の傾きのばらつきそのもの。

- ・ ただし N はテンプレートの Y 軸ではなく Z 軸に対応している。tube の既定 (Y軸向き) のまま複製すると、複製の軸と N の角度が全200個すべて正確に 90.00度、つまり完全に横倒しになる。テンプレートを Z軸向きに作り直すと全200個すべて 0.00度で完全に一致した。ばらつきがゼロなので、これは偶然ではなく仕様。

- ・ tube SOP の orient の既定は Y軸。そのため何も考えずに tube を複製すると必ず横倒しになる。地面に立てたいなら、テンプレートを Z軸向きに作るか、あとで回転させる必要がある。

- ・ pscale は大きさだけを変える。向きのばらつきは 10.84度 / 31.69度 のまま完全に同じで、大きさが 0.45 固定から 0.16~0.63 に広がった。向きと大きさは独立して制御できる。

- ・ tube SOP の type の既定は Primitive で、1点 / 1面の「本物の円柱プリミティブ」になる。ポリゴンとして扱いたいなら Polygon に変える必要がある。これに気づかず、最初は1点のテンプレートを200個複製していた (200点になるだけで何も見えない) 。

- ・ 測り方の失敗も1つ記録しておく。最初は「重心から最も遠い点」で複製の向きを測ろうとしたが、円錐の底面リングの点がすべて等距離のため、選ばれる点が複製ごとにばらついて 21.44度 という無意味な値が出た。固定の点番号を使う方式に変えたら 0.00度 になった。等距離の点が並ぶ形では「最も遠い点」は一意に決まらない。

- ・ 実用の型はこうなる。地面 → normal → scatter → copy to points。テンプレートは Z軸向きに作る。大きさを変えたいなら散布点に pscale を持たせる。この4ノードで「地面に物を生やす」が完成する。

次にやること

- ・ 複製の向きに、傾きだけでなくランダムな回転を混ぜる (up ベクトルの扱い)
- ・ density や面積のアトリビュートで散布密度に偏りをつける
- ・ for-each で複製ごとに違う処理をかける (実験010の予定)

HeightField で地形を作る — 高さを面ではなく数の並びで持つ

2026-09-24 | Houdini 21.0.700 | /obj/terrain | ノード 4 / 接続 3

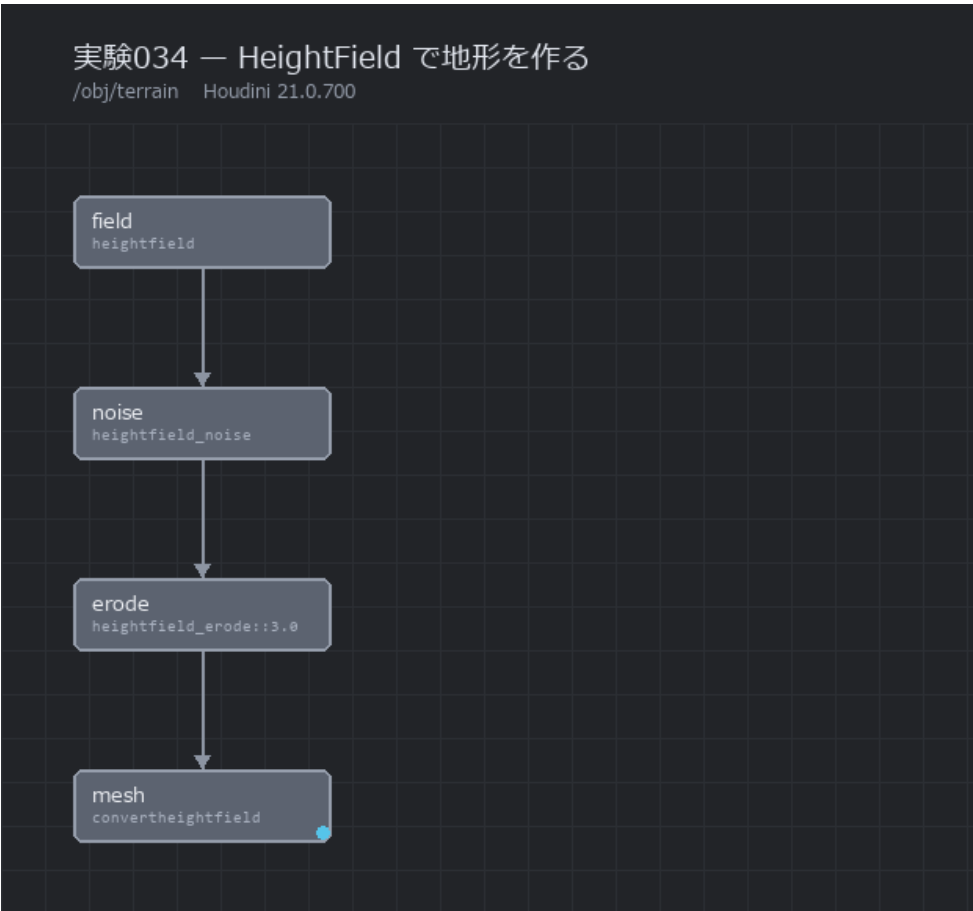
何を作ったか

実験007では grid に mountain をかけて地形を作った。点を並べて高さをずらす、ポリゴンのやり方である。Houdini にはもう1つ HeightField という仕組みがあり、こちらは専用のノードが45種もある。

違いは持ち方にある。HeightField は高さを2次元のボリュームとして持つ。面も点も持たず、プリミティブはたった2つ（height と mask）。そのぶん、ポリゴンでは難しい浸食（erode）のような計算ができる。

測って分かったこと。解像度は $\text{size} \div \text{gridspacing}$ で決まり、Grid Samples という名前のパラメータは 128 にしても 1024 にしても解像度を1ミリも動かさない。浸食はレイヤーを2枚から7枚に増やし、勾配を32.7%下げる。amp は純粋な倍率で、4通り試して $\text{幅} \div \text{amp}$ が小数点以下6桁まで同じだった。

ノードの接続



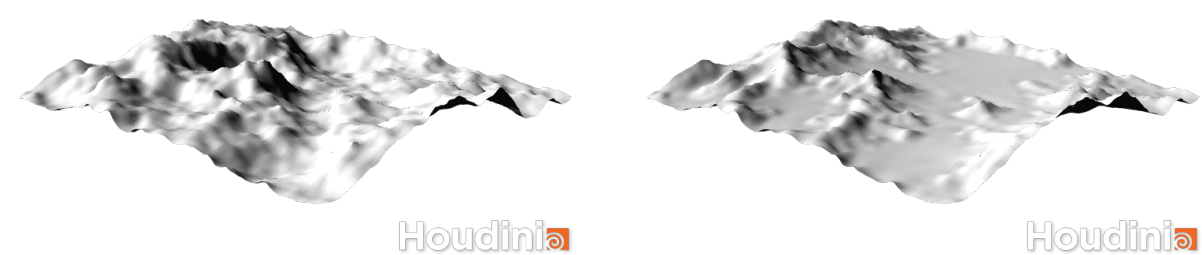
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
field	heightfield	size _x = 200.0 size _y = 200.0

ノード	タイプ	デフォルトから変更したパラメータ
noise	heightfield_noise	amp = 30.0 elementsiz = 80.0
erode	heightfield_erode::3.0	folder0 = 1 folder1 = 1 iterations = 2 seed = 0.5160613059997559
mesh	convertheightfield	(変更なし)

浸食のあるなし

同じノイズから作った地形。画は起伏を見せるため amp を 120 にしている（amp は純粋な倍率なので形は変わらない）。カメラは2枚とも同じ。



ノイズをかけただけ。丸い起伏が並ぶ。

浸食を4回。尾根と谷ができ、山肌に筋が出る。

調べたこと	結果
HeightField の中身	2点 / 2プリミティブ (height と mask のボリューム)
高さの解像度	100×100×1
メッシュに直すと	10,000点 / 9,801面
点の数と解像度	100×100 = 10,000 でぴったり一致

1. 解像度は何が決めるか

size と gridspacing を変えて、できた高さボリュームの解像度を読んだ。

size	gridspacing	解像度	size ÷ gridspacing
200	2.0	100×100	100
200	1.0	200×200	200
200	4.0	50×50	50
400	2.0	200×200	200
100	2.0	50×50	50

2. Grid Samples は効くのか

名前からすると解像度を決めそうだが、size 200 / gridspacing 2 のまま値だけ変えた。

gridsamples	解像度
128	100×100
512（既定）	100×100
1024	100×100

3. 浸食は何を変えるか

heightfield_erode の Iterations per Frame を変えた。解像度は 100×100。

回数	かかった秒	レイヤー数	最低	最高	平均	勾配
0（かけない）	0.04	2	-3.747	4.993	0.330	0.07893
1	0.68	7	-2.626	4.926	0.288	0.06710
2	0.80	7	-2.619	4.728	0.255	0.06124
4	2.35	7	-2.609	4.615	0.196	0.05315

4. amp は高さの倍率か（実験006と同じ問い）

heightfield_noise の Amplitude だけを変えて、高さの幅を測った。

amp	高さの幅	幅 ÷ amp
7.5	2.1850	0.291329
15.0	4.3699	0.291329
30.0	8.7399	0.291329
60.0	17.4797	0.291329

わかったこと

- ・ HeightField はプリミティブが2つしかない。height と mask という2枚のボリウムで、点は2個。同じ地形をメッシュに直すと 10,000点 / 9,801面になる。同じ情報を、まったく違う入れ物で持っている。
- ・ 点の数は解像度そのものだった。100×100 の高さボリウムをメッシュにすると、ちょうど 10,000点。マス目1つが点1つになる。
- ・ 解像度は $\text{size} \div \text{gridspacing}$ で決まる。5通り試してすべて一致した。細かくしたいなら gridspacing を下げる。
- ・ Grid Samples は効かなかった。128 にしても 1024 にしても 100×100 のまま。名前からは解像度を決めそうに見えるが、動かない。この2つが並んでいるのが、最初につまずくところになる。
- ・ 浸食はレイヤーを2枚から7枚に増やす。増えるのは debris（がれき）・sediment（堆積）・flow（流れ）・flowdir.x・flowdir.y（流れの向き）。削った結果だけでなく、その過程も残る。あとで色を塗り分けるのに使える。
- ・ 浸食は地形をなだらかにした。勾配は 0.07893 から 0.05315 へ、32.7%下がった。見立てどおり。
- ・ 削って、積もっている。最高は 4.993 → 4.615 と下がり、最低は -3.747 → -2.609 と上がった。山が削れて谷が埋まる。平均も 0.330 → 0.196 と下がっており、全体として土が低いところへ移っている。

- ・時間は回数に比例しない。1回で 0.68秒、2回で 0.80秒、4回で 2.35秒。1回目に下ごしらえの費用がかかり、そのあとは増えていく。100×100 という小さな地形でこれなので、実用の解像度では相応に重い。

- ・amp は純粋な倍率だった。7.5 / 15 / 30 / 60 の4通りで、幅÷amp が 0.291329 と小数点以下6桁まで同じ。実験006で mountain の height について確かめたことが、HeightField 側でも同じように成り立つ。

- ・ただしamp の数字がそのまま高さになるわけではない。amp 30 で実際の幅は 8.74。0.29倍ほどに収まる。狙った高さにしたいなら、この比率を測ってから逆算する。

次にやること

- ・浸食のレイヤー (debris / sediment / flow) で色を塗り分ける
- ・heightfield_scatter で地形の上に物を生やす (実験008の応用)
- ・035 以降、まだ触っていない分野に進む

地形の上に生やし分ける ― 浸食が残した情報で木の場所を決める

2026-09-24 | Houdini 21.0.700 | /obj/terrain | ノード 10 / 接続 11

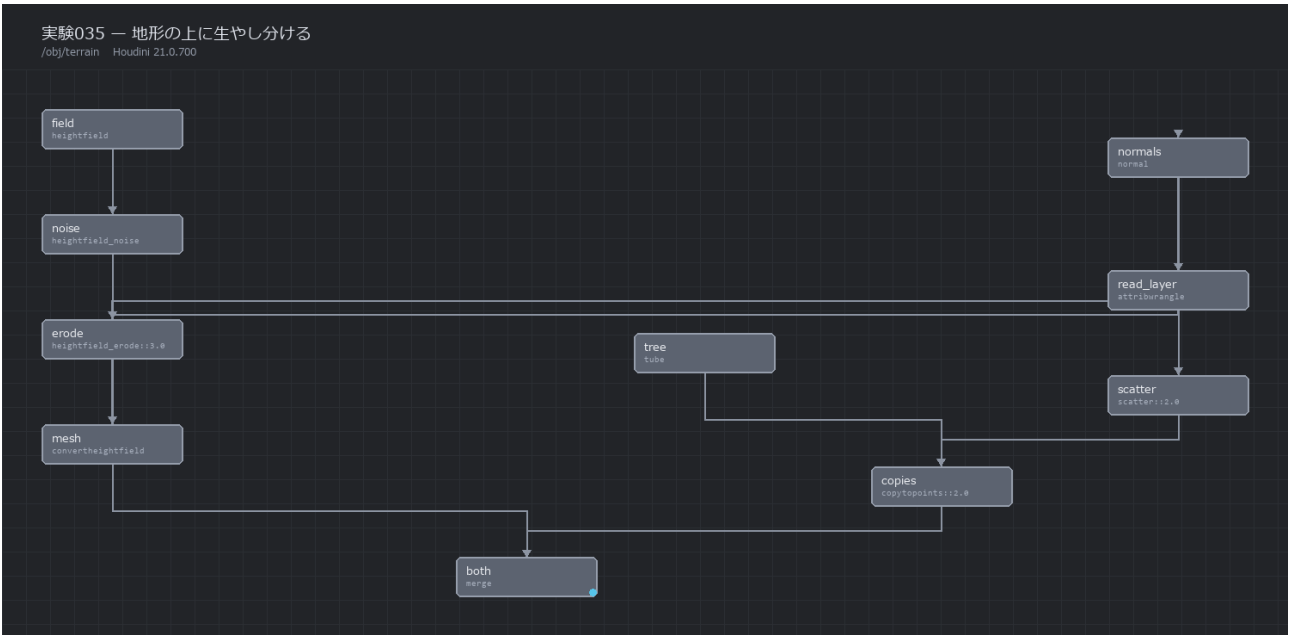
何を作ったか

実験008では平らな地面に木を200本立てた。どこに立てるかは完全に運任せで、地形の形とは関係がなかった。

実験034で、浸食が過程をレイヤーとして残すことが分かった。がれき・堆積・流れ。このうち sediment（堆積）は「土が溜まったところ」を表しているはずなので、これを散布の重みに使えば、崖には生えず平らなところに生える地形が自動で作れる。

狙いどおりになった。まず sediment 自体が、緩いところに急なところの 170.64倍溜まっている。これを重みに使うと、生えた場所の勾配の平均が 0.06474 から 0.00332 へ、94.9%下がった。木は崖を避け、土の溜まった谷筋に集まる。

ノードの接続



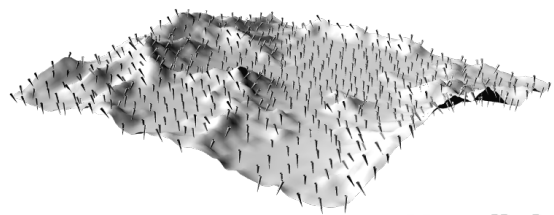
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
field	heightfield	sizeX = 200.0 sizeY = 200.0
noise	heightfield_noise	amp = 120.0 elementsSize = 80.0
erode	heightfield_erode::3.0	folder0 = 1 folder1 = 1 iterations = 4 seed = 0.2596234083175659
mesh	convertheightfield	(変更なし)
normals	normal	(変更なし)

ノード	タイプ	デフォルトから変更したパラメータ
read_layer	attribwrangle	snippet = f@layer = volumesample(1, "sediment", @P); // 勾配も点ごとに持たせる。あとで「どこに生えたか」を測るのに使う vector up = set(0, 1, 0); f@slope = 1.0...
scatter	scatter::2.0	densityattrib = layer npts = 600 usedensityattrib = 1
tree	tube	cols = 8 height = 6.0 rad1 = 0.6 rad2 = 0.05 rx = 90.0 type = 1
copies	copytopoints::2.0	(変更なし)
both	merge	(変更なし)

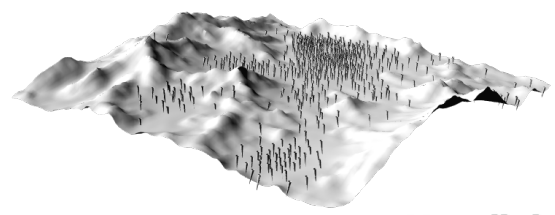
重みのあるなし

同じ地形・同じ本数（600本）。違いは「どこに置くかの重み」だけ。カメラは2枚とも同じ。



Houdini

重み無し。崖も尾根も関係なく、一様にばらまかれる。



Houdini

sediment を重みに。崖が裸になり、谷筋に集まる。

段取り	使うノード
地形を作って浸食する	heightfield → heightfield_noise → heightfield_erode
メッシュに直す	convertheightfield
面の向きを作る	normal（これが無いと複製が回らない）
レイヤーを点に読み込む	attribwrangle で volumesample
重みを付けて散布	scatter の密度アトリビュートに指定
木を複製する	copy to points

1. 浸食が残したレイヤーの中身

100×100 のマス目それぞれが持つ値。「値のあるマス」は 0 でないマスの割合。

レイヤー	最小	最大	平均	値のあるマス
height（高さ）	-10.4156	18.1462	0.6104	49.1%
mask	0.0000	0.0000	0.0000	0.0%
debris（がれき）	0.0000	2.5569	0.0452	32.5%

レイヤー	最小	最大	平均	値のあるマス
sediment（堆積）	0.0000	11.0788	0.8346	49.5%
flow（流れ）	0.0000	4.1504	0.8009	100.0%
flowdir.x	-4.1177	2.9576	-0.1818	39.4%
flowdir.y	-2.2426	4.0191	0.0780	50.6%

2. sediment は本当に「緩いところ」に溜まっているか

メッシュの点 10,000個を、勾配の中央値で2つに分けて比べた。

どちら	sediment の平均
急なほう（勾配が中央値より上）	0.00973
緩いほう（勾配が中央値より下）	1.65957
緩いほうは急なほうの	170.64倍

3. 重みを付けると、生える場所はどう変わるか

どちらも600本。生えた場所の勾配と sediment の値を測った。

	本数	勾配の平均	勾配の中央値	sediment の平均
重み無し	600	0.06474	0.03674	0.75115
重みあり	600	0.00332	0.00063	4.23520
変化	—	-94.9%	-98.3%	5.64倍

わかったこと

- ・ sediment は緩いところに 170.64倍 溜まっていた。勾配の中央値で2つに分けると、急なほうの平均が 0.00973、緩いほうが 1.65957。浸食が「どこに土が溜まったか」をそのまま記録している。

- ・ 重みを付けると、生える場所の勾配が 94.9% 下がった。平均 0.06474 → 0.00332、中央値は 0.03674 → 0.00063 で 98.3% 下がる。木は崖を避ける。

- ・ 生えた場所の sediment の値は 5.64倍になった。0.75115 → 4.23520。狙ったレイヤーの値が高いところへ、はっきり寄っている。

- ・ mask は最初から最後まで全部ゼロだった。最小も最大も 0.0000。heightfield が作る2枚のうち1枚は、何か塗らないかぎり空のままである。「レイヤーがある」と「値が入っている」は別の話。

- ・ flow だけは 100% のマスに値がある。他のレイヤーは 32~51% なのに対し、流れはどこにでもある。地形全体をなでる量なので、散布の重みには向かない。使うレイヤーは、値の散らばり方を見てから選ぶ。

- ・ レイヤーを点に持ってくるのは volumesample 1行。f@layer = volumesample(1, "sediment", @P); と書き、2本目の入力に浸食後の HeightField をつなぐ。実験026で炎の温度を読んだのと同じやり方が、そのまま地形にも効く。

- ・ normal は scatter の前に置く。実験008・030で確かめたとおり、scatter は N を作らない。引き継ぐだけなので、上流で作っておかないと木が全部同じ向きに立つ。

- ・ 木のテンプレートは tube を X に 90度回して Z 軸向きにしている。copy to points はテンプレートの Z 軸を N に合わせる（実験008）。回さないと全部横倒しになる。

次にやること

- flow の向き (flowdir) を使って、草の流れに沿って寝かせる
- debris と sediment で色を塗り分ける
- 036 以降、まだ触っていない分野に進む

流れの向きに草を寝かせる — flowdir は x と y が入れ替わっている

2026-09-24 | Houdini 21.0.700 | /obj/terrain | ノード 11 / 接続 12

何を作ったか

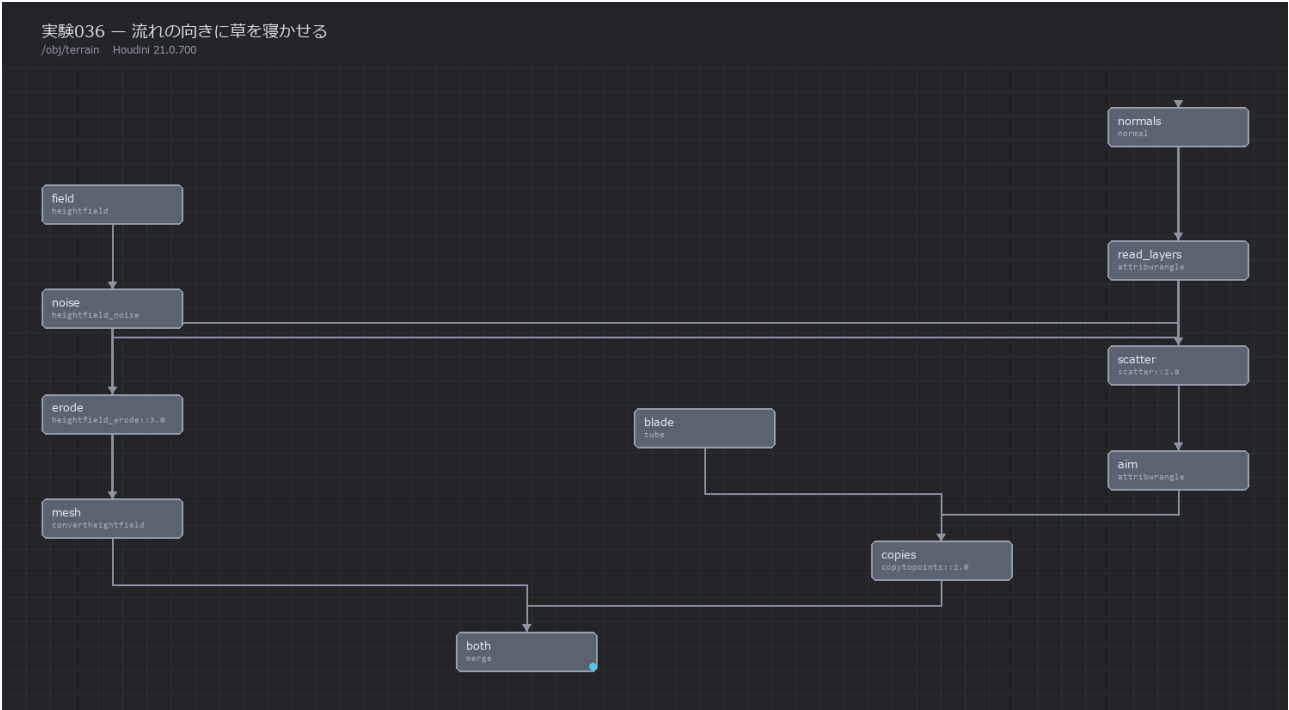
実験035では浸食のレイヤーのうち sediment（堆積）だけを使った。残っているのが flowdir.x と flowdir.y、水が流れた向きである。名前からすると下り坂の向きのはずだが、確かめていない。

素直に書いたら外れた。flowdir.x を3Dのx、flowdir.y を3Dのz に入れると、高さから作った下り坂の向きとの角度が平均96.48度。でたらめ（90度）とほとんど変わらない。

そこで x と y の割り当てを8通り総当たりで測った。1つだけ明らかに違う値が出た。入れ替えた (fy, 0, fx) が平均39.95度・中央29.94度で、30度未満が50.1%を占める。x と y は入れ替えて使う。

向きが決まれば、あとは N に入れるだけ。複製の軸と狙った向きのずれは900本すべてで0.000度、真上からの傾きは0度から90度になった。草はびたりと寝る。

ノードの接続



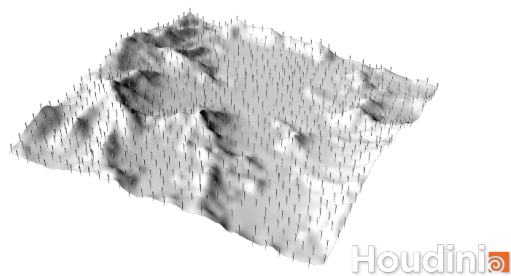
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
field	heightfield	size_x = 200.0 size_y = 200.0
noise	heightfield_noise	amp = 120.0 elements_size = 80.0
erode	heightfield_erode::3.0	folder0 = 1 folder1 = 1 iterations = 4 seed = 0.35988283157348633
mesh	convertheightfield	(変更なし)

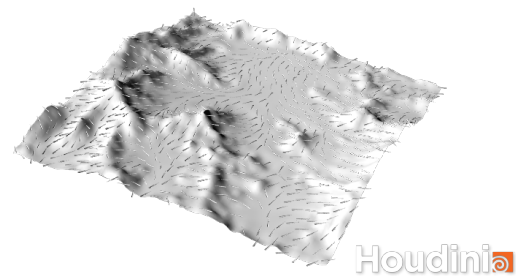
ノード	タイプ	デフォルトから変更したパラメータ
normals	normal	(変更なし)
read_layers	attribwrangle	snippet = f@layer = volumesample(1, "sediment", @P); float fx = volumesample(1, "flowdir.x", @P); float fy = volumesample(1, "f...
scatter	scatter::2.0	npts = 900
aim	attribwrangle	snippet = if (length(v@flowvec) > 1e-6) { @N = normalize(v@flowvec); } else { @N = set(0, 1, 0); }
blade	tube	cols = 6 height = 7.0 rad1 = 0.35 rad2 = 0.02 rx = 90.0 type = 1
copies	copytopoints::2.0	(変更なし)
both	merge	(変更なし)

向きのあるなし

同じ地形・同じ900本。違いは N に何を入れるかだけ。カメラは2枚とも同じ。



N は真上。草はまっすぐ立ち、向きの情報が無い。



N に流れの向き。草が寝て、水の通り道が筋になって見える。

調べたこと	結果
メッシュの点	10,000個
流れのある点	9,996個 (99.96%)
草	900本
convertheightfield が持ち込む点アトリビュート	debris, flow, flowdir_x, flowdir_y, height, mask, sediment, uv

1. x / y の割り当てを8通り総当たり

flowdir.x と flowdir.y を、3D の x と z にどう割り当てるか。高さから作った下り坂の向きとの角度を測った。でたらめなら、どれも平均90度になる。

割り当て	平均	中央値	判定
(fx, 0, fy) 素直な並び	96.48度	99.55度	でたらめと同じ
(-fx, 0, -fy)	83.52度	80.45度	でたらめと同じ

割り当て	平均	中央値	判定
(fy, 0, fx) 入れ替え	39.95度	29.94度	これが正しい
(-fy, 0, -fx)	140.05度	150.06度	逆向き
(fx, 0, -fy)	93.42度	94.01度	でたらめと同じ
(-fx, 0, fy)	86.58度	85.99度	でたらめと同じ
(fy, 0, -fx)	90.56度	88.58度	でたらめと同じ
(-fy, 0, fx)	89.44度	91.42度	でたらめと同じ

2. 正しい割り当てでの、下り坂との一致具合

(fy, 0, fx) で測った角度の分布。0度なら完全に下り坂を向いている。

角度	その範囲に入る割合
30度未満	50.1%
60度未満	78.5%
90度未満	90.5%
120度未満	95.3%
平均 / 中央値	39.95度 / 29.94度

3. 複製は、入れた向きを向くか

草1本ごとに、その軸と N の角度を測った。900本すべて。

N に入れたもの	狙いとのずれ（平均）	狙いとのずれ（最大）	真上からの傾き	ばらつき
真上 (0,1,0)	0.000度	0.000度	0.00度	0.00
流れの向き	0.000度	0.000度	90.00度	0.00

わかったこと

- ・素直に書くと外れる。flowdir.x を3Dのx、flowdir.y を3Dのzに入れると、下り坂との角度が平均96.48度。でたらめ（90度）とほとんど変わらない。
- ・x と y は入れ替えて使う。8通り試して、set(fy, 0, fx)だけが平均39.95度・中央29.94度まで下がる。他の7通りはすべて83~140度で、でたらめか逆向きだった。1つだけ明らかに外れた値が出るので、迷わず決められる。
- ・総当たりが効いた。「合わないから諦める」のでも「たぶんこうだろうと決め打つ」のでもなく、ありうる並びを全部試して数字で選んだ。8通りしかないので、総当たりが一番速い。
- ・正しい割り当てなら、半分为30度未満に入る。50.1%が30度未満、90.5%が90度未満。完全に一致はしないが、これは浸食の流れが局所の傾きだけで決まらない（上流から流れてきた勢いが残る）ためと考えられる。ここは確かめていないので断定しない。
- ・複製は入れた向きをぴたりと向いた。900本すべてで、狙いとのずれが 0.000度。真上を入れれば0度（立つ）、流れの向きを入れれば 90度（寝る）。ばらつきも 0.00。
- ・草の軸の測り方でつまづいた。最初は複製の最初の点と最後の点を結んで軸にしたが、ぴったり180度ずれた。tube は2つの輪でできていて、輪の並びが想定と逆だった。ずれが「ちょうど180.000度」なので、誤差ではなく向きの取り違いだと分かる。

- ・ 下り坂の向きは、法線からは作れなかった。normal SOP は N を点ではなくバーテックスに書く（実験030で確認済み）。点のラングルから @N を読むと空になる。高さのボリュームから直接、隣との差で作直した。

- ・ convertheightfield はレイヤーを点アトリビュートとして持ち込む。メッシュの時点で flow や sediment が点に付いている。同じ名前で書こうとすると衝突する。最初 v@flow と書いて、既にある float の flow とぶつかり、ベクトルのつもりが数値1つになっていた。

次にやること

- ・ debris と sediment で地形を塗り分ける
- ・ 草の長さを流れの強さで変える
- ・ 037 以降、まだ触っていない分野に進む

heightfield_noise の Amplitude は高さの幅ではない — 幅は Amplitude の 23~29%、Center Noise を切ると Amplitude の半分だけ上がる

2026-09-24

何を作ったか

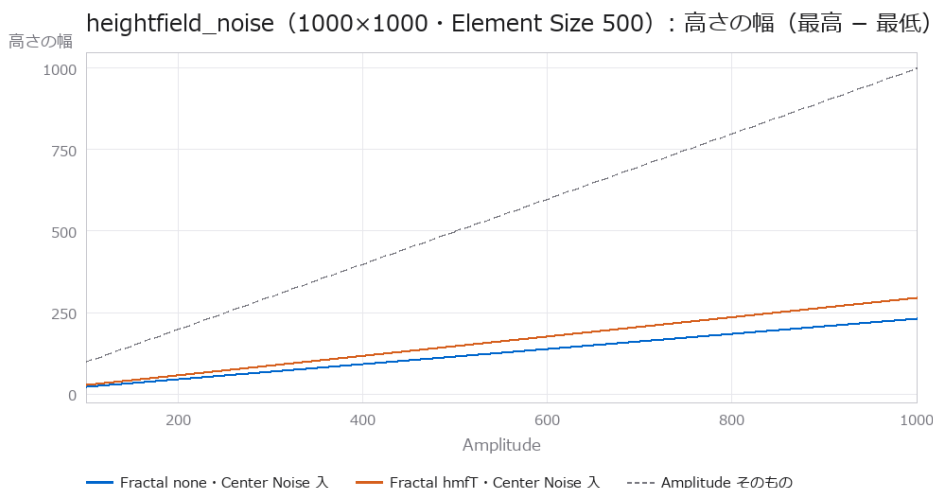
1000 × 1000 の heightfield (Grid Spacing 4、250 × 250 の升) に heightfield_noise (Combine = Replace、Element Size 500、Noise Type は既定) をかけ、height の升の値から最小・最大・平均・標準偏差を数えた。

**高さの幅 (最高 - 最低) は Amplitude よりずっと小さい。 **Fractal なしで Amplitude の 23.2%、既定の hmfT で 29.4%。 Amplitude 1000 でも、山と谷の差は 232~294。この大きさの地面 (Element Size と同じくらいの範囲) では、ノイズの山の高いところまで届かないとみられるが、確かめていない。

**高さは Amplitude にぴったり比例する。 **幅 ÷ Amplitude、標準偏差 ÷ Amplitude は、100・500・1000 のどれでも4桁同じ (標準偏差は Amplitude の 4.94%・5.36%)。同じ模様を縦に伸ばしているだけ。

**Center Noise を切ると、全体がちょうど Amplitude の半分だけ上がる。 **平均の差は 50, 250, 500, 50, 250, 500 (Amplitude 100・500・1000 の順、Fractal なし・hmfT)。入れると平均が 0 のまわりに来る (ぴったり 0 ではない)。

Fractal・Center Noise・Amplitude と高さ



高さの幅は、点線 (Amplitude) の 1/4 ほどの傾きの直線になる。

Fractal	Center Noise	Amplitude	最低	最高	幅	幅 ÷ Amplitude	平均	標準偏差 ÷ Amplitude	秒
none	入	100	-12.378	10.783	23.161	0.2316	-0.634	0.0494	0.337
none	入	500	-61.891	53.914	115.805	0.2316	-3.172	0.0494	0.204

Fractal	Center Noise	Amplitude	最低	最高	幅	幅 ÷ Amplitude	平均	標準偏差 ÷ Amplitude	秒
none	入	1000	-123.781	107.828	231.61	0.2316	-6.344	0.0494	0.203
none	切	100	37.622	60.783	23.161	0.2316	49.366	0.0494	0.205
none	切	500	188.109	303.914	115.805	0.2316	246.828	0.0494	0.203
none	切	1000	376.219	607.828	231.61	0.2316	493.656	0.0494	0.208
hmfT	入	100	-12.577	16.813	29.39	0.2939	0.583	0.0536	0.224
hmfT	入	500	-62.884	84.067	146.95	0.2939	2.916	0.0536	0.225
hmfT	入	1000	-125.767	168.134	293.901	0.2939	5.831	0.0536	0.228
hmfT	切	100	37.423	66.813	29.39	0.2939	50.583	0.0536	0.226
hmfT	切	500	187.116	334.067	146.95	0.2939	252.916	0.0536	0.224
hmfT	切	1000	374.233	668.134	293.901	0.2939	505.831	0.0536	0.228

わかったこと

- ・欲しい高さの幅から Amplitude を決めるなら、約4倍（hmfT なら約3.4倍）にする。この大きさ・Element Size のとき。
- ・Amplitude を倍にすると、地形はそのまま縦に倍。形は変わらない。
- ・Center Noise を切ると、地面が Amplitude/2 だけ持ち上がる。海面の高さを決めるときに効く。

次にやること

- ・Element Size を変えたときの幅 ÷ Amplitude
- ・heightfield_erode で高さの合計（土の量）が保たれるか

heightfield_erode は土の量を保たない — 40 フレームで height の平均が 5 下がり、削れた分の大半は地形から消える

2026-09-24

何を作ったか

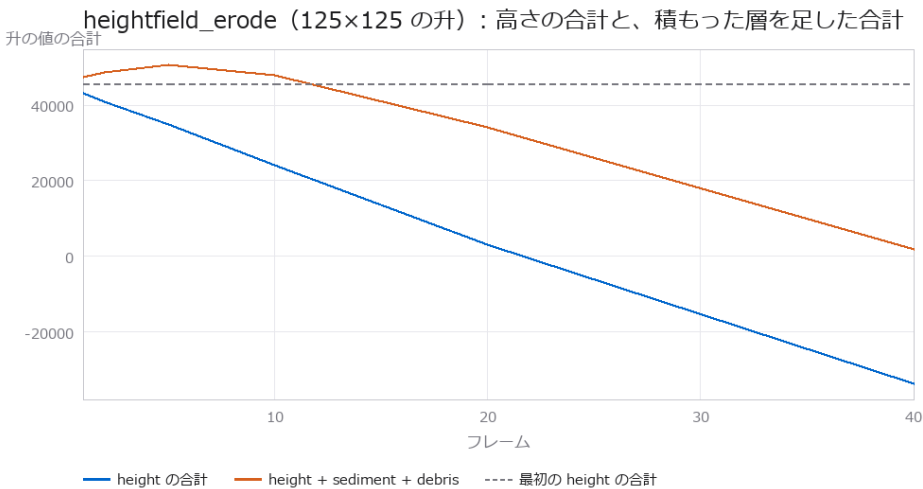
1000 × 1000 の heightfield (125 × 125 の升) に heightfield_noise (Amplitude 500) で山を作り、heightfield_erode (既定。Freeze は切った) をフレーム 1 から 40 まで進めた。height と、erode が作る層 (sediment・debris など) を升ごとに足し合わせた。

**height の合計は減り続けた。 **最初は 45,591、フレーム 40 で -33,644。 升1つあたりの平均では 2.92 → -2.15 (-5.07)。 最高点は 84.74 → 72.09、最低点は -62.81 → -39.19 と、山は低く、谷は浅くなった。

**削れた分を sediment と debris に足しても、元の量にならない。 **3つを足した合計は、フレーム 5 まではむしろ最初より多く (50,577)、そのあと減って、フレーム 40 で 1,805 (最初の 4%)。 流れる水と一緒に端から出ていく、か、層の値が「量」ではなく「濃さ」を表している、かのどちらかとみられるが、確かめていない。

**速い。 **125 × 125 の升で、40 フレームを 3.8 秒。

フレームごとの合計



青 (height) は直線的に減り、橙 (層を足したもの) も途中から下がる。

フレーム	経った秒	height の合計	平均	最低	最高	sediment	debris	3つの合計	flow
1	1.11	43,047	2.75	-57.07	84.74	3,727	529	47,304	3,300
2	1.40	40,937	2.62	-54.15	84.74	6,947	748	48,632	6,119
5	1.59	34,771	2.23	-47.48	84.21	14,655	1,151	50,577	13,657
10	1.89	24,075	1.54	-43.7	82	22,453	1,298	47,826	24,056
20	2.52	3,032	0.19	-40.23	78.68	29,872	1,199	34,103	40,290
40	3.77	-33,644	-2.15	-39.19	72.09	33,990	1,459	1,805	64,396

わかったこと

- ・ erode をかけると、地形全体が低くなる。40 フレームで平均 5 ほど。海面や建物の高さを決めるのは erode のあと。
- ・ 山は削れ、谷は埋まる。最高点も最低点も真ん中へ寄る。
- ・ sediment ・ debris の値を「土の量」として足しても合わない。

次にやること

- ・ 端を閉じたとき（Border の扱い）に量が保たれるか
- ・ sediment の値の単位

heightfield_noise

の高さの幅は、地面に入る模様の数で決まる — Element Size を地面の 1/20 にすると Amplitude の 6 割

2026-09-24

何を作ったか

実験168 の続き。1000 × 1000 の heightfield に heightfield_noise (Amplitude 1000、Center Noise 入) をかけ、Element Size を 50~2000 と変えて、高さの幅 (最高 - 最低) ÷ Amplitude を測った。

**地面に模様がたくさん入るほど、幅は Amplitude に近づく。 **Fractal なしで、地面の幅 ÷ Element Size が 0.5 なら 7%、1 なら 18%、2 (実験168) なら 23%、4 なら 30%、10 なら 55%、20 なら 62%。模様が少ないと、ノイズの高い山や深い谷に当たる前に地面が終わる。

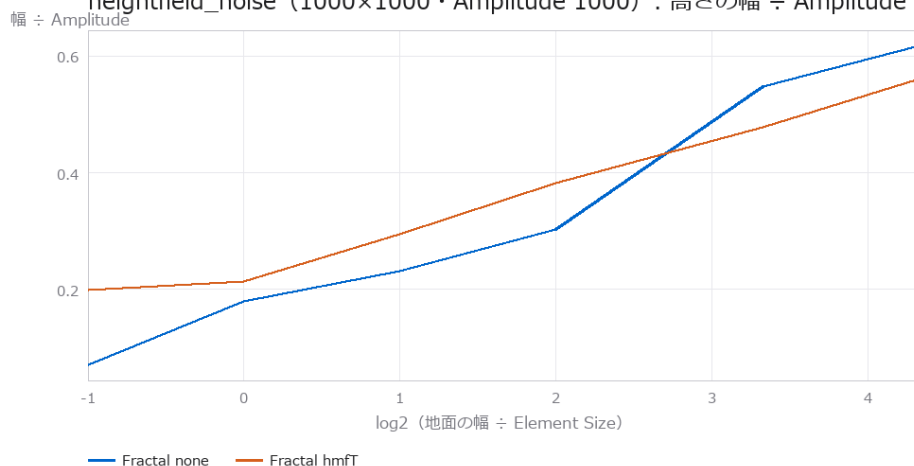
標準偏差 ÷ Amplitude は、模様が 10 個を超えると約 7% で落ち着く (10 で 6.9%、20 で 7.2%)。幅は広がり続けるが、ならした高さのばらつきは頭打ちになる。

**hmfT (既定) も同じ傾向。 **模様がが多いときは Fractal なしより幅が少し狭い (20 個で 56%)、少ないときは広い。

**20 個入っても、幅は Amplitude の 6 割どまり。 **Amplitude は「山と谷の差」ではなく、ノイズの値に掛ける数とみられる。

Element Size と高さ

heightfield_noise (1000×1000・Amplitude 1000) : 高さの幅 ÷ Amplitude と、



模様数を倍にするたびに幅が広がる。

Fractal	Element Size	地面 ÷ Element Size	幅 ÷ Amplitude	標準偏差 ÷ Amplitude	最低	最高	平均	秒
none	50	20	0.6170	0.0716	-264.4	352.6	-2.83	0.333
none	100	10	0.5475	0.0687	-245.7	301.8	0.85	0.206
none	250	4	0.3031	0.0567	-123.8	179.3	14.89	0.204
none	500	2	0.2316	0.0494	-123.8	107.8	-6.34	0.205

Fractal	Element Size	地面 ÷ Element Size	幅 ÷ Amplitude	標準偏差 ÷ Amplitude	最低	最高	平均	秒
none	1000	1	0.1801	0.0319	-85.2	94.9	-25.87	0.204
none	2000	0.5	0.0713	0.0126	-55.5	15.8	-39.14	0.206
hmfT	50	20	0.5603	0.0664	-275.7	284.6	-6.37	0.230
hmfT	100	10	0.4786	0.0655	-217.2	261.4	-1.95	0.225
hmfT	250	4	0.3823	0.0593	-135.5	246.8	11.54	0.223
hmfT	500	2	0.2939	0.0536	-125.8	168.1	5.83	0.233
hmfT	1000	1	0.2141	0.0452	-86.1	128	-11.7	0.228
hmfT	2000	0.5	0.1987	0.0446	-70.2	128.5	-20.71	0.226

わかったこと

- ・欲しい高さの幅から Amplitude を決めるときは、Element Size も一緒に見る。同じ Amplitude でも幅は 7~62%。
- ・ Element Size を小さくすると、同じ Amplitude でも山が高くなる。細かい起伏を足したつもりが、高さも変わる。
- ・ 平均も揺れる。模様が少ないと、Center Noise 入でも平均が 0 から数十ずれる。

次にやること

- ・ 地面の広さを変えずに、点の細かさ（Grid Spacing）を変えたとき
- ・ heightfield_remap で幅をそろえる